

OKTS: Open Knowledge Tool Search (Progressive Tool Disclosure with Portable Descriptors)

PrabhuKiran Ganapavarapu

Independent Researcher

prabhukiran.g@gmail.com

ORCID: [0009-0001-9971-9589](https://orcid.org/0009-0001-9971-9589)

August 7, 2026

Abstract

Large language model (LLM) agents are increasingly given access to hundreds of tools drawn from many sources (Model Context Protocol servers, plain functions, HTTP/OpenAPI endpoints, sub-agents). Loading every tool’s JSON schema into the context window up front is the dominant pattern, but its cost scales with the size of the catalog rather than the difficulty of the task, and is re-paid on every turn: schemas consume a large fraction of the window before the task begins, and near-duplicate tools across sources (`github.create_issue` vs. `gitlab.create_issue` vs. `linear.create_issue`) blur together and induce wrong selections. The common alternative—retrieval-augmented generation (RAG) over tool embeddings—removes the up-front cost but replaces it with standing *infrastructure*: an embedding model to host or an API to call, a vector store to maintain, and a re-embedding step on every catalog change. We present **OKTS**, a source-agnostic runtime that (i) describes each tool as a portable, git-versioned Markdown descriptor (the **OKT** format) and (ii) serves any catalog behind a fixed three-tool interface (`search_tools`, `load_tool`, `call_tool`) via *progressive disclosure*. On a corpus of 148 tools adapted from 20 widely used MCP servers, OKTS reduces per-query tool-schema tokens by **94.1%** relative to the load-everything baseline, a reduction that grows with corpus size. For ranking, OKTS uses a fully offline, *zero-infrastructure* hybrid retriever—BM25 combined with a deterministic feature-hashing embedding that needs no model, API, or vector store—which lifts top-1 tool-selection accuracy from 92.7% to 97.6% and eliminates near-duplicate collisions (95.1% $\rightarrow$ 100.0%) over a strong BM25 baseline, at essentially identical token cost. An ablation is reported faithfully: the gain comes from *fusing* the two lexical signals—neither BM25 nor the hashing embedding reaches it alone—and not from the category-hierarchy prefilter or graph expansion, which are accuracy-neutral on this corpus and instead serve to keep the returned-result count fixed and to surface alternatives. All numbers are produced by a reproducible harness over an open corpus.

1 Introduction

The prevailing way to give an LLM agent a tool is to place that tool’s name, description, and full JSON input schema into the model’s context as a function-calling definition. This works for a handful of tools and fails at scale for two independent reasons.

Context cost. Every tool definition is paid for in tokens on every turn, whether or not the tool is used. With the standardization of the Model Context Protocol (MCP) [1], it is now routine for an

agent to be wired to many servers at once, each exposing tens of tools; the combined schemas can consume a large fraction of the context window before the task begins, starving the prompt and diluting attention across definitions the agent will never call [4].

Selection cost. As the catalog grows, it accumulates *near-duplicate* tools—especially across sources that wrap the same capability. A query like “open a bug ticket in my GitHub repo” must select `github.create_issue` over the lexically almost-identical `gitlab.create_issue` and `linear.create_issue`. Flat lexical matching frequently picks the wrong sibling or hallucinates parameters.

The community response has largely been *retrieval over tools*: index tool descriptions, retrieve the top- k for each query, and expose only those. When the index is a dense vector store, this is RAG applied to tools [8, 9]. It solves the context-cost problem but introduces a standing *operational* cost that is easy to overlook: a semantic index requires an embedding model (hosted locally or behind an API), a vector store to persist and query, and a re-embedding pass whenever a tool is added, edited, or removed. For a tool catalog—which changes far more often than a document corpus and is small enough that heavyweight retrieval infrastructure is disproportionate—this is a poor trade.

Contributions. We describe **OKTS** (Open Knowledge Tool Search), a runtime that keeps the benefit of retrieval while removing both the up-front context cost *and* the embedding infrastructure:

1. **The OKT descriptor** (§3): a portable, human-readable, git-versioned Markdown file per tool, whose YAML frontmatter is grouped by the runtime phase that consumes it. It is the durable artifact a semantic index lacks—diffable, editable, and reviewable.
2. **Progressive disclosure** (§4): a fixed three-tool interface (`search_tools/load_tool/call_tool`) that returns lightweight refs, loads exactly one schema on demand, and dispatches the call—so tool-schema context is bounded and, empirically, near-constant regardless of catalog size (94.1% reduction at 148 tools, growing with the corpus).
3. **Zero-infrastructure hybrid retrieval** (§4, §6): BM25 fused with a *deterministic feature-hashing embedding* [12] that needs no model, no API, and no vector store, yet improves top-1 selection accuracy and eliminates near-duplicate collisions versus a strong BM25 baseline.
4. **An honest empirical study** (§6): a reproducible harness over an open 148-tool corpus, including an ablation that shows which structural signals help selection accuracy and which do not.

A companion observation frames the comparison: swapping in a *learned* embedder is supported (an injectable `embed_fn`) but would reintroduce exactly the model/API/vector-store cost OKTS exists to avoid; we therefore benchmark the zero-infrastructure default deliberately.

2 Related Work

Tool-using LLMs. ReAct [13] and Toolformer [11] established interleaving reasoning with tool calls. Scaling the number of tools motivated retrieval-based selection: Gorilla [8] retrieves API documentation, and ToolLLM/ToolBench [9] operate over thousands of real APIs with a retriever front-end. These systems typically rank tools with a dense retriever, inheriting the embedding-infrastructure cost discussed above.

Retrieval over tools to reduce prompt bloat. The closest work retrieves a small, relevant subset of tools per query instead of loading the whole catalog. RAG-MCP [3] mitigates prompt bloat in LLM tool selection by retrieving MCP tools with a retriever before invocation. Toolshed [5] scales tool-equipped agents through “RAG-Tool Fusion” over a tool knowledge base. OKTS shares their retrieve-then-use premise but differs on three axes: the retrieved unit is a *portable, git-versioned descriptor* (the OKT file) rather than an opaque vector; the runtime is source-agnostic—MCP, functions, HTTP, sub-agents, and search—behind one three-tool interface; and the default retriever needs *no* embedding model, API, or vector store, so adding or editing a tool never triggers a re-embedding pass.

Retrieval. Our ranker builds on BM25 [10] and, for the hybrid variant, feature hashing (the “hashing trick”) [12] as a deterministic, model-free embedding; score fusion follows standard weighted-sum and reciprocal-rank-fusion practice [2]. Token counts use the `c1100k_base` BPE encoding [7].

3 The OKT Descriptor

An **OKT** (Open Knowledge Tool) file is YAML frontmatter followed by a Markdown body. The format is a *profile* of the Open Knowledge Format (OKF) [6], a portable Markdown-with-frontmatter convention for representing organizational knowledge: OKT specializes it for a single callable tool (`type: tool`). The frontmatter is emitted in groups, each labeled with the runtime phase that consumes it, so an opened file is self-documenting and a code review of a tool change is meaningful. Listing 1 is the verbatim output of the runtime’s serializer for one corpus tool.

Listing 1: An OKT descriptor, exactly as the runtime serializes it. Frontmatter is grouped by consuming phase; the body is retrieval prose, never the calling contract. The **alternatives** edges were derived by the auto-linker, not declared by the source.

```
---
# identity
type: tool
id: github.search_issues
title: Search Issues

# match -- ranked by search_tools (phase 1); never sent at call time
description: Search issues and pull requests across repositories
  by free-text query.
tags:
- github
- mcp

# call -- loaded by load_tool (phase 2); the calling contract
input_schema:
  type: object
  required:
  - query
  properties:
    query:
      type: string

# route -- used by call_tool to dispatch (phase 3)
interface: mcp
```

```

target: github
side_effects: read
invocation: async

# graph edges -- expanded during search_tools
alternatives:
- github.create_issue
- github.get_issue
- github.list_issues
- github.update_issue
---

Synonyms: search, issues, github, mcp.

Use this to search issues and pull requests across repositories
by free-text query.

Gotcha: This is read-only and safe to call speculatively; it
does not modify state.

```

An optional `auth` field names a credential (e.g. `auth: github_token`); the runtime resolves it at dispatch through a pluggable secrets provider—by default the environment variable `OKTS_SECRET_GITHUB_TOKEN`—so the value never appears in the descriptor or in agent context.

The format enforces five invariants. (1) The agent-facing surface is always exactly three tools.¹ (2) `input_schema` is always a structured JSON Schema, so arguments can be validated before dispatch. (3) The body is *retrieval prose*, used only for ranking—never the calling contract. (4) Credentials are resolved inside the runtime at dispatch and never enter agent context. (5) A minimal descriptor requires only six fields (`type`, `id`, `title`, `description`, `input_schema`, `interface`); everything else is optional and additive. A structural conformance check enforces exactly these rules before a bundle is served—the six required fields present and non-empty, the `type: tool` discriminator, `input_schema` structured rather than prose, `interface/side_effects/invocation` drawn from their known value sets, and every graph edge resolving to a concept in the same bundle. We validate the OKT profile’s own rules; we do not claim to validate against a normative OKF schema, which is not published as a machine-checkable artifact.

Descriptors are produced by adapters—one per source type (`mcp`, `function`, `http`, `agent`, `search`)—and then *auto-linked*: a query-independent pass derives a `server/resource` category hierarchy and `alternatives` edges between near-duplicate tools from the flat set. The resource noun is the tool name’s last non-stopword `_`-token, plural-stemmed (`list_issues` → `issue`), and same-group members are linked as mutual alternatives, capped at 5 per tool for determinism.

4 Progressive Disclosure and Retrieval

4.1 The three-tool interface and the token model

Regardless of catalog size, the agent sees three tools: `search_tools(query, k)` returns up to k lightweight refs (`id`, `title`, `description`—*no* schema); `load_tool(id)` returns the one structured `input_schema` for the chosen tool; and `call_tool(id, args)` validates the arguments against that schema and dispatches to the real source.

¹`call_tool` additionally has an `acall_tool` coroutine form for awaiting asynchronous targets. It is the same phase-3 tool under a different calling convention, not a fourth tool, and hosts expose one or the other.

We account for tokens as an agent would pay them. Because function-calling APIs re-send the tool definitions with every request, both quantities below are per-query costs and are directly comparable. The *baseline* loads all N tool schemas up front: $C_{\text{raw}} = \sum_{i=1}^N \text{tok}(\text{schema}_i)$, where each schema_i is the tool’s name, description, and JSON parameter schema serialized as one function definition. Under OKTS the per-query cost is

$$C_{\text{OKTS}} = \underbrace{C_{\text{meta}}}_{3 \text{ tools}=305} + \sum_{j=1}^k \text{tok}(\text{ref}_j) + \text{tok}(\text{schema}_{\text{loaded}}), \quad (1)$$

i.e. the constant three-meta-tool schemas, the k lightweight refs, and the *single* schema loaded for the chosen tool. Crucially C_{meta} and the number of refs do not depend on N , so C_{OKTS} is bounded as N grows while C_{raw} grows linearly. Token counts use tiktoken `cl100k_base` (with a deterministic `chars/4` fallback when the encoder is unavailable).

4.2 Ranking: hybrid, then structure

The baseline retriever is BM25 [10] ($k_1 = 1.5$, $b = 0.75$) over each descriptor’s match text—the concatenation of **description**, **tags**, and **body**. The full retriever fuses BM25 with a dense score and then applies two structural steps.

Deterministic hashing-dense. Rather than a learned embedder, the default dense signal hashes features into a 256-dimensional L2-normalized vector via the hashing trick [12]: each whitespace token and each character n -gram ($n \in \{3, 4, 5\}$, weighted 0.5 relative to whole tokens) is mapped to a signed bucket by a SHA-1 digest, so the mapping is stable across processes and machines. Scoring is a dot product, which equals cosine similarity on the normalized vectors. It is fully deterministic, requires no model download or network call, and gives partial credit for morphological variants (**issue/issues**) that a plain BM25 tokenizer treats as unrelated. One asymmetry is deliberate: the dense side embeds only a concept’s *own* **description** and **tags** and excludes the **body**, so it describes what a tool *is* rather than what it mentions in passing. The motivating case is a hand-authored body that steers a reader away from a sibling (“don’t confuse this with **list_issues**”), which as raw text is lexically indistinguishable from an endorsement; BM25 keeps the **body**, the dense signal does not. The two scores are min-max normalized and combined by weighted sum with equal weights (0.5/0.5); reciprocal rank fusion is also implemented [2] and is not used in the reported configuration. This is the entire “embedding” cost of OKTS: no model, no API, no vector store, and nothing to re-embed when a tool changes.

Hierarchy prefilter and graph expansion. Two optional structural steps operate over the auto-linked bundle. The hierarchy prefilter scores each category path by its stemmed token overlap with the query, normalized by path length, and adds a flat +0.15 to members of the top 2 categories; nothing is ever removed (a harder *restrict* mode exists but is off by default), so this step cannot push the full retriever below the baseline. Graph expansion pulls up to 3 graph neighbors per top-ranked hit, damped relative to their seed— $\times 0.85$ across an **alternatives** edge and $\times 0.6$ across **composes_with/prerequisites**. On this corpus the auto-linker derives only **alternatives** edges, so only that damping is exercised. Importantly, expansion is constrained to a *fixed result count*: because every sibling is damped strictly below its seed, siblings compete *within* the k returned refs (the merged set is re-sorted and truncated to k) rather than being appended past it. Consequently the top-1 result is always a rank hit, and the returned-ref count—and hence the per-query token cost—stays fixed. We emphasize that this is a *result-count* constraint in code; the token flatness in Eq. 1 is a measured consequence, not an enforced token limit.

5 Experimental Setup

Corpus. We assemble 148 tools across 20 widely used MCP servers (GitHub, GitLab, Linear, Postgres, Supabase, BigQuery/GCP, MongoDB, Redis, Qdrant, Docker, Kubernetes, AWS, Cloudflare, Vercel, Notion, Figma, Sentry, Playwright, Chrome DevTools, filesystem). *The tool lists are hand-authored approximations of each server’s published surface*—realistic names and compact schemas with the cross-server near-duplicates that make selection hard—not live captures; the ingestion path is identical to a live capture (only the source of the `tools/list` payload differs).

Each `tools/list` payload passes through the ordinary offline build: `adapt` $\rightarrow$ `enrich` $\rightarrow$ `auto-link` $\rightarrow$ `validate`. We stress that the enrichment step is a *deterministic offline* enricher, not an LLM: it emits a fixed template of synonyms drawn from the tool’s own id and tags, a restatement of the description, and a side-effect gotcha (visible in the body of Listing 1). The retrieval text is therefore mechanically derived, which is the conservative setting—an LLM enrichment pass, which the runtime also supports, would add genuine synonyms and would if anything raise absolute accuracy.

The auto-linker derives 91 categories, of which 60 contain a single tool (median category size 1, maximum 6), and 204 **alternatives** edges covering 88 of the 148 tools. It derives no `composes_with/prerequisites` edges from a flat `tools/list`, so graph expansion here operates purely over **alternatives**.

Queries. A labeled set of 41 natural-language queries, each mapping to one correct tool id plus a list of **distractors**: the wrong-but-tempting near-duplicates the retriever must not prefer. There are 71 distractors in total (mean 1.7 per query, and every query has at least one), of which 61 name a tool on a *different* server than the correct answer—the cross-server collision failure mode—and the remainder a sibling on the same server.

Metrics. We report `accuracy@1` and `accuracy@k` (is the correct tool ranked first / in the top k), mean reciprocal rank (MRR), and *collision-avoidance*: the fraction of queries in which the correct tool outranks every one of its listed distractors that appears in the returned set.² We report token cost alongside every accuracy number, never one without the other: the raw all-schemas cost, the average per-query OKTS cost (Eq. 1), and the reduction. We use $k = 5$ throughout.

6 Results

6.1 Selection accuracy and token cost

Table 1 compares the flat BM25 baseline against the full retriever. The full retriever raises `accuracy@1` from 92.7% to 97.6%, raises MRR from 0.963 to 0.988, and eliminates near-duplicate collisions (95.1% $\rightarrow$ 100.0%), at essentially identical per-query token cost ($\approx$ 531 tokens). Both retrievers reach 100.0% and 100.0% `accuracy@k` respectively: on this corpus the correct tool is always *within* the top 5, so the differentiation is entirely at the top rank and in collision avoidance. This also bounds how much a better ranker could gain here—the headroom is a re-ordering problem, not a recall problem.

²The implementation also counts a query as avoided when neither the correct tool nor any distractor is returned at all, which would make the metric vacuously satisfiable. That branch is never taken here: `accuracy@k` is 100.0% for every configuration in Tables 1 and 2, so the correct tool is always present and every query is scored against at least one distractor (70/71 distractor ids resolve to a tool in the corpus).

Table 1: Flat BM25 baseline vs. the full OKTS retriever on 148 tools, 41 labeled queries, $k = 5$. Token cost is per query.

Retriever	acc@1	acc@ k	MRR	coll. avoid.	avg tok/qry
Flat BM25 (baseline)	92.7	100.0	0.963	95.1	532
Graph-aware (ours)	97.6	100.0	0.988	100.0	531

6.2 What actually drives the gain (ablation)

Table 2 isolates each signal. The result is reported as measured, including where it is null. The accuracy gain comes *entirely from the hybrid (BM25 + hashing-dense) ranking*: “Hybrid, no structure” already reaches 97.6% accuracy@1 and 100.0% collision-avoidance—the full retriever’s numbers exactly. Adding the hierarchy prefilter or graph expansion on top does not further improve accuracy@1 or collision-avoidance on this query set, and neither structural signal alone moves the BM25 baseline (92.7% and 92.7% accuracy@1, versus 92.7% for BM25 alone).

Three honest readings follow. First, the effect is one of *complementarity*, not of either signal being stronger: BM25 alone and hashing-dense alone both sit at 92.7%/92.7% accuracy@1, and only their fusion reaches 97.6%. The dense signal does carry collision-avoidance on its own (100.0%), which is consistent with its intended role—giving morphological partial credit that exact-token BM25 misses on near-duplicate names. Second, no learned embedding is involved anywhere in this gain. Third, the hierarchy is far too fine-grained on this corpus to discriminate: 60 of the 91 derived categories contain exactly one tool (median size 1), so boosting the top 2 categories most often boosts the single tool that was already winning. Graph expansion, in turn, is *designed* not to alter the top rank, since siblings are damped strictly below their seed. Their value is preserving the fixed return count while surfacing alternatives—and, plausibly, scoping corpora larger or flatter than this one—not lifting accuracy@1 on this set. We do not have evidence for the larger-corpus claim and do not assert it.

Table 2: Ablation. The accuracy gain is attributable to hybrid ranking; the structural signals are accuracy-neutral here (see text).

Configuration	acc@1	acc@ k	MRR	coll. avoid.	avg tok/qry
BM25 only (baseline)	92.7	100.0	0.963	95.1	532
Dense only (hashing)	92.7	100.0	0.957	100.0	530
Hybrid, no structure	97.6	100.0	0.988	100.0	531
BM25 + hierarchy	92.7	100.0	0.963	95.1	532
BM25 + graph	92.7	100.0	0.963	95.1	530
Hybrid + hierarchy + graph	97.6	100.0	0.988	100.0	531

6.3 Token reduction grows with the corpus

Figure 1 sweeps corpus size. Sub-corpora are prefixes of the full bundle in its canonical (server-alphabetical) order, re-auto-linked at each size, with the same 41 queries run against each; only the token accounting is read off this sweep, never accuracy, since a truncated corpus does not contain every query’s answer. Because C_{OKTS} is bounded (Eq. 1) while C_{raw} grows linearly, the reduction climbs from a modest fraction on a tiny corpus to 94.1% at the full 148 tools (88.6% already at 75

tools), crossing 85% at roughly 60 tools. The per-query OKTS cost stays near-constant—within a 521–531-token band across the entire sweep—while the raw cost grows linearly (Figure 2). This is the sense in which large token reductions are a *large-corpus property* of progressive disclosure. The converse is worth stating plainly: on a small catalog the fixed 305-token meta-tool overhead dominates and progressive disclosure *costs* tokens rather than saving them. Measured on this corpus, OKTS breaks even at roughly ten tools (−15% at 8 tools, +14.9% at 11); below that an agent should simply be handed every schema.

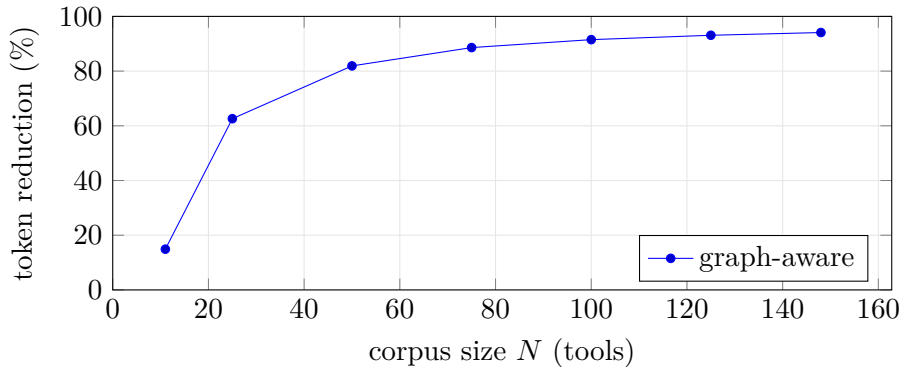


Figure 1: Per-query tool-schema token reduction vs. corpus size.

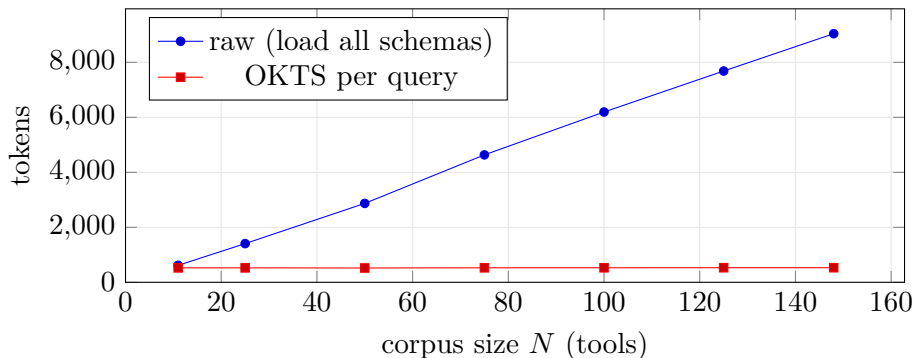


Figure 2: Absolute token cost: the load-everything baseline grows linearly; the OKTS per-query cost stays near-constant.

7 The Infrastructure Comparison

The quantitative evaluation above compares against a strong *lexical* baseline. The more consequential comparison for practice is architectural, against embedding-RAG tool search. Table 3 summarizes the asymmetry. OKTS attains near-ceiling selection accuracy and 94.1% token reduction while requiring *no* embedding model, *no* inference API in the query path, *no* vector store, and *no* re-embedding when a tool changes—the index is portable text, and an edit re-ranks immediately. A learned embedder can be injected for a possible further lift, but doing so reintroduces precisely these costs; we regard the zero-infrastructure configuration as the default and the learned embedder as an optional upgrade, and we benchmark the former. The table compares *standing operational*

requirements, which are architectural facts, not measured quantities; we make no claim about relative ranking quality here (see §8).

Table 3: Architectural comparison. OKTS keeps retrieval’s benefit without the standing infrastructure of embedding-RAG.

Dimension	Embedding-RAG tool search	OKTS (default)
Index artifact	opaque float vectors	portable OKT Markdown (git-diffable)
Add / edit a tool	re-embed (model / API)	edit text; re-rank immediately
Query-path dependency	embedding model reachable	none (lexical + hashing)
Vector store	required	none
Explainability	cosine score	edge provenance + inspectable text

8 Limitations and Threats to Validity

Retrieval-level, not end-to-end. We measure whether search ranks the correct tool, not whether an LLM completes the downstream task better. End-to-end agent success is left to future work (it requires an agent loop and API budget).

Hand-authored corpus and mechanical retrieval text. The 20 tool lists are realistic approximations of published surfaces, not live captures; exact tool counts and parameters will differ by server version. The bodies are produced by the deterministic offline enricher rather than written by hand or by an LLM, so the retrieval prose is thinner than a production bundle’s would be. The adapter path is identical for live captures, but absolute numbers should be read as indicative.

Small, author-labeled query set. With 41 queries near a 97.6% ceiling, a single query is worth 2.4 points of accuracy@1; the full retriever’s margin over the baseline is two queries, and its perfect collision-avoidance score rests on two. Effect sizes are small, we report no significance tests, and the labels are our own. A larger, independently labeled set would sharpen the accuracy claims.

Structural signals are unvalidated at scale. The ablation shows the hierarchy prefilter and graph expansion to be accuracy-neutral here, and we attribute that to a corpus whose derived categories are mostly singletons. That is an explanation, not evidence: we have not tested a corpus where the hierarchy is coarse enough for the prefilter to discriminate, so the case for these signals rests on the fixed result count and on surfacing alternatives, both of which are mechanical properties rather than measured wins.

No live embedding-RAG baseline. We compare to BM25 quantitatively and to embedding-RAG only architecturally (§7); we do not claim to beat a tuned semantic retriever on ranking quality—only to match a strong lexical baseline and reach high absolute accuracy without its infrastructure.

9 Conclusion

OKTS shows that an LLM agent can be given a large, multi-source tool catalog while seeing only three tools, paying a near-constant token cost (a 94.1% reduction at 148 tools), and selecting the

right tool with high accuracy using a fully offline, zero-infrastructure retriever. The tool’s durable unit is a portable, reviewable descriptor rather than an opaque vector, and the ranking gain over a strong lexical baseline comes for free from a deterministic hashing signal—no embedding model, API, or vector store to maintain. We report our ablation honestly, including that the category hierarchy and graph expansion are accuracy-neutral on this corpus and instead serve to fix the returned-result count and surface alternatives. We hope the OKT descriptor and the accompanying open harness make multi-source tool serving easier to build, evaluate, and trust.

A Reproducibility

Every number in this paper is produced by `paper/make_results.py`, which builds the corpus and runs the same `okts.eval` harness the test suite uses, then writes the tables and plot data this document `\inputs`. To reproduce:

```
pip install -e "[serve,dense]" tiktoken
python -m okts.eval.corpus --k 5 # headline compare + reduction sweep
python paper/make_results.py # regenerate all tables/figures
```

The `dense` extra installs `numpy`, which the hybrid retriever requires; `tiktoken` is optional but without it token counts fall back to the `chars/4` heuristic and will not match the figures reported here. The corpus lives in `eval/corpus/` and the labeled queries in `eval/corpus_queries.yaml`. Every quantity in the running text—corpus and query-set statistics, retriever hyperparameters, and each ablation row cited by name—is emitted as a $\text{\LaTeX}$ macro by the same script, so no number in this paper is typed by hand. Token counts use `tiktoken cl100k_base`. Code: <https://github.com/prabhukirangit/OKTS>.

References

- [1] Anthropic. Model context protocol. <https://modelcontextprotocol.io>, 2024. Open specification.
- [2] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *ACM SIGIR Conference on Research and Development in Information Retrieval*, 2009.
- [3] Tiantian Gan and Qiyao Sun. RAG-MCP: Mitigating prompt bloat in LLM tool selection via retrieval-augmented generation. <https://arxiv.org/abs/2505.03275>, 2025. arXiv:2505.03275.
- [4] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.
- [5] Elias Lumer, Vamse Kumar Subbiah, James A. Burke, Pradeep Honaganahalli Basavaraju, and Austin Huber. Toolshed: Scale tool-equipped agents with advanced RAG-tool fusion and tool knowledge bases. <https://arxiv.org/abs/2410.14594>, 2024. arXiv:2410.14594.
- [6] Sam McVeety and Amir Hormati. Introducing the Open Knowledge Format. Google Cloud Blog, June 2026. <https://cloud.google.com/blog/products/data-analytics/how-the-open-knowledge-format-can-improve-data-sharing>; reference implementation: <https://github.com/GoogleCloudPlatform/knowledge-catalog>.

- [7] OpenAI. tiktoken: A fast BPE tokeniser for use with OpenAI’s models. <https://github.com/openai/tiktoken>, 2022.
- [8] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive apis. *arXiv preprint arXiv:2305.15334*, 2023.
- [9] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, et al. ToolLLM: Facilitating large language models to master 16000+ real-world apis. In *International Conference on Learning Representations (ICLR)*, 2024.
- [10] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.
- [11] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [12] Kilian Weinberger, Anirban Dasgupta, John Langford, Alex Smola, and Josh Attenberg. Feature hashing for large scale multitask learning. In *International Conference on Machine Learning (ICML)*, 2009.
- [13] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.